

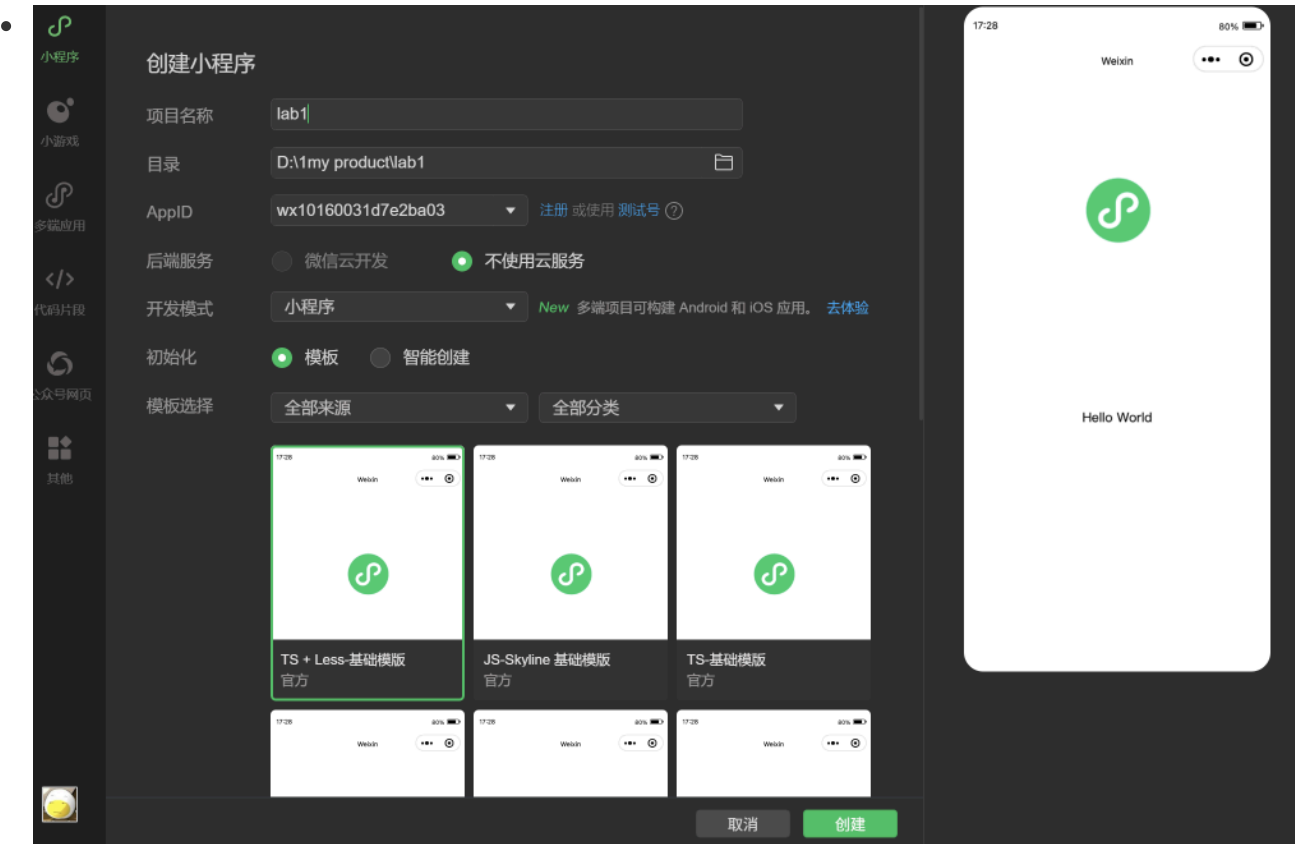
一、实验内容

1.环境搭建与项目创建：

- 下载并安装最新稳定版的微信开发者工具。
- 在本地创建一个空文件夹用于存放项目文件。
- 打开微信开发者工具，点击“+”号新建项目。填写项目名称，选择刚才创建的空文件夹作为目录。
- 在 AppID 处选择“测试号”，不使用云服务。特别注意在模板选择中，选择“不使用模板”，以创建一个干净的初始项目。

2.目标一：学习使用快速启动模板创建小程序

- 在新建项目时，选择“使用模板”。
- 项目创建后，观察工具自动生成的丰富目录结构（包含 `pages/index`、`pages/logs`、`utils` 等）。体验模板自带的基础功能（如获取用户信息、查看启动日志），了解快速启动模板如何帮助开发者迅速搭建具备通用功能的项目脚手架。



3.目标二：学习不使用模板手动创建小程序

- 在 `pages/index/index.wxml` 文件中构建页面骨架，使用 `<div>` 等标签组织内容，并通过 `{{}}` 语法绑定动态数据。同时，添加一个 `<button>` 按钮组件，并使用 `bindtap` 属性绑定点击事件。

```

<view class="container">
  <view class="header">
    <text class="title">{{mainTitle}}</text>
    <text class="subtitle">人格修正模拟器</text>
  </view>

  <!-- 添加 <button> 按钮，并绑定 enterPlan 点击事件 -->
  <view class="action-area">
    <button class="start-btn" bindtap="enterPlan">开启第 1 天</button>
  </view>
</view>

```

- 在 pages/index/index.wxss 文件中编写样式代码，例如设置背景颜色、元素居中等，美化页面视觉效果。

```

/* 设置页面整体背景颜色 */
page {
  background-color: #E0F7FA;
  height: 100%;
}

/* 使用 Flex 布局实现元素居中 */
.container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
}

```

- 在 pages/index/index.js 文件中编写页面逻辑。在 data 对象中定义初始变量，并在绑定的点击事件函数中使用 this.setData() 方法更新变量的值，从而实现界面文字的动态切换效果。

```

enterPlan: function () {
  console.log("按钮被点击了");

  // 测试任务
  if (this.data.mainTitle === '暑假重塑计划') {
    this.setData({
      mainTitle: 'day1'
    });
  } else {
    this.setData({
      mainTitle: '暑假重塑计划'
    });
  }
  // 提示弹窗
  wx.showToast({
    title: '标题已切换',
    icon: 'none',
    duration: 1000
  });
}

```

```
}
```

二、问题总结与体会

描述实验过程中所遇到的问题，以及是如何解决的。有哪些收获和体会，对于课程的安排有哪些建议。

1. 遇到的问题及解决过程

问题1：界面修改后未实时生效

- **描述：**在手动编写 WXML 结构后，模拟器上的界面没有任何变化。
- **解决：**检查发现是因为修改后未执行保存操作。在开发者工具中，必须使用快捷键 `Ctrl + S` 保存文件，工具才会自动重新编译并刷新模拟器界面。

问题2：数据已修改但视图层未更新

- **描述：**在编写 JS 事件时，起初直接尝试用赋值语句 `this.data.message = '你好，世界！'` 来改变文字，发现界面毫无反应。
- **解决：**复习文档后得知，小程序的数据绑定机制要求必须使用 `this.setData()` 方法。通过此方法更新数据，底层框架才会捕捉到变化并同步渲染到 WXML 视图层。

2. 实验收获与体会

- **对比两种创建方式的优劣：**通过本次实验，我直观体会到了两种开发起步方式的不同。“快速启动模板”能一键生成包含常用工具类（如 `utils.js`）和多页面的框架，适合直接进入业务开发；而“不使用模板”从零开始搭建，则剥离了所有干扰项，让我最纯粹、最深刻地认识了 WXML、WXSS 和 JS 这三剑客的最基础调用关系。
- **掌握核心通讯机制：**在纯手工编写“Hello World”的过程中，我深刻记住了 `this.setData()` 的重要性。这是逻辑层向视图层发送更新指令的标准方式，是后续开发所有复杂交互的基石。